

group by

The **GROUP BY** statement groups rows that have the same values into summary rows, like "find the number of customers in each country".

The **GROUP BY** statement is often used with aggregate functions (**COUNT()**, **MAX()**, **MIN()**, **SUM()**, **AVG()**) to group the result-set by one or more columns.

use company;

*select * from department;*

select count(id),department_name from department group by department_name;

select count(id),department_name from department group by department_name order by count(id) desc ;

select sum(salary_amount),department_name from department group by department_name;

HAVING

The **HAVING** clause was added to SQL because the **WHERE** keyword cannot be used with aggregate functions.

select sum(salary_amount),department_name from department group by department_name where sum(salary_amount)>111111; -- wrong

select sum(salary_amount),department_name from department group by department_name having sum(salary_amount)>111111;

subquery

A subquery, also known as a nested query or inner query, is a SQL query embedded within another SQL statement. Here's a brief note on subqueries:

- Definition: A subquery is a query nested within another SQL query. It's enclosed within parentheses and used to retrieve or manipulate data based on the result of the outer query.
- Types of Subqueries:
 - Scalar Subquery: Returns a single value and can be used wherever an expression is allowed.
 - Multi-row Subquery: Returns multiple rows and can be used with operators like IN, ANY, ALL, etc.
- Purpose:
 - Subqueries allow for complex data retrieval and manipulation by breaking down a problem into smaller, more manageable parts.
 - They enable dynamic filtering, sorting, and aggregation based on conditions derived from other tables or the result of other queries.
- Syntax:
 - Subqueries are enclosed within parentheses and can appear in various parts of SQL statements such as SELECT, INSERT, UPDATE, DELETE, or as part of a FROM clause.

[COMPLETE DATA STUDENT](#)

? to display name, location, phone_number of the students from data table whose section is a

? delete data in students table who are from salem?

? update the phone number of students to 100 of section b

1.-- ? To display NAME, LOCATION, PHONE_NUMBER of the students from data table whose section is A

Select NAME, LOCATION, PHONE_NUMBER from complete_data
WHERE ROLL_NO IN
(SELECT ROLL_NO from STUDENT where SECTION="A");

2. -- ? DELETE DATA OF STUDENTS FROM STUDENTS TABLE WHO ARE FROM SALEM?

```
DELETE FROM student WHERE ROLL_NO IN ( SELECT ROLL_NO
    FROM complete_data
    WHERE LOCATION = "SALEM");
```

```
SELECT * FROM STUDENT;
```

3. -- ? UPDATE THE PHONE NUMBER OF STUDENTS TO 100 OF SECTION B

```
UPDATE complete_data
SET PHONE_NUMBER=100
WHERE ROLL_NO IN ( SELECT ROLL_NO
    FROM STUDENT
    WHERE SECTION IN ("A","B"));
```

VIEW

- Definition: A view in SQL is a virtual table derived from the result set of a SELECT query. It doesn't store data on its own but represents a stored query that can be referenced like a table.
- Purpose:
 - Views simplify complex queries by encapsulating them into a single, reusable object.
 - They provide a layer of abstraction, allowing users to work with data in a simplified manner without needing to understand the underlying complexity of the query.
- Characteristics:
 - Views can be queried, updated, inserted into, or deleted from, depending on the permissions granted.
 - They are stored in the database as metadata, not as actual data.
 - Views can join and simplify multiple tables, aggregate data, or present a subset of data to users.
 - Views can provide a consistent interface to data, even if the underlying schema changes.

-- VIEW

```
CREATE VIEW MY_VIEW AS
```

```
SELECT *
```

```
FROM complete_data
```

```
WHERE LOCATION="CHENNAI";
```

```
SELECT * FROM MY_VIEW;
```

```
ALTER VIEW MY_VIEW AS SELECT *
```

```
FROM complete_data
```

```
WHERE LOCATION="SALEM";
```

```
SELECT * FROM MY_VIEW;
```

```
DROP VIEW MY_VIEW;
```

```
SELECT * FROM MY_VIEW;
```

CREATE TABLE FROM VIEW

```
CREATE TABLE new_table AS
```

```
SELECT * FROM MY_VIEW;
```